OKSBDC Chatbot — Full Codebase Report

Repository: `Oklahoma-SBDC/OKSBDCChatbot` · **Branch reviewed:** `jaysonhauschildoksbdc-cuddly-potato`
Generated: 2026-08-12 **Scope:** Full read of server (`server/`) and client (`client/`) source, existing docs (`server/docs/`), test suite, dependency audit, and git history.

---

0. Urgent — Read This First

While reviewing the working-tree diff on `server/docs/configuration.md`, this review found that a **real, currently-valid dashboard password was committed in plaintext** to git history (commit `63ffc79`, "Add dashboard account authentication") and is only now being scrubbed from the *working copy* — it remains fully recoverable from `git log` / `git show` regardless of this edit.

This review went one step further and **cryptographically confirmed the password is live**: it matches the PBKDF2 hash stored for the `mark.barnett` account in `server/data/dashboard-accounts.json`, which is itself tracked in git (real usernames, real salts, real hashes, all committed). The password value itself is intentionally **not repeated anywhere in this report** to avoid creating a second exposure.

This is an active incident, not a historical finding. Recommended action, in order:

1. **Rotate the `mark.barnett` dashboard password immediately** (`npm run dashboard:create-user` doesn't currently support rotation in place — see §7 — so this may require a manual account edit or a new script; at minimum, deactivate/delete the account and recreate it).
2. Rotate `JWT_SECRET` as a precaution — any token signed with the current secret should be treated as suspect once the account tied to it is touched.
3. Treat `server/data/dashboard-accounts.json` as sensitive and stop committing it (see §7.1 for remediation options, including history rewrite considerations — that step needs your explicit sign-off since it rewrites shared history).
4. Audit dashboard audit logs (`server/logs/*.jsonl` , `dashboard_rule_added` and related events) for any actions taken under `mark.barnett` 's account that weren't authorized.

Everything else in this report is normal engineering feedback; this section is not.

---

1. Executive Summary

OKSBDC Chatbot is a two-part web application: a **React/Vite widget + admin dashboard** (`client/`) embedded on the Oklahoma SBDC website, backed by an **Express/Node API** (`server/`) that classifies and answers small-business questions using Google's Gemini models, grounded by a locally crawled **RAG (retrieval-augmented generation)** knowledge base of oksbdc.org content.

The codebase is larger and more sophisticated than a typical "chatbot widget" — it includes a full authentication system, a regex-based safety/message-filter engine with a dashboard rule editor, an audit-logged rollback system for prompt/rule changes, temporary IP banning with rolling violation windows, RAG context caching with preloading, and layered LLM-safety heuristics that override the model's own judgment for personalized legal/tax/financial questions. This is meaningfully above the bar for a hobby project — it reflects real production hardening effort.

At the same time, the review surfaced one active credential-exposure incident (§0), several concrete bugs, a handful of exploitable gaps (regex-DoS surface, spoofable ban/rate-limit identity, an environment-variable footgun that can silently disable ban enforcement), and some architectural debt (very large route files, no shared state across horizontally-scaled instances, no CI pipeline). None of the code-level issues are exotic — they're the kind of thing a focused week of hardening work resolves.

Codebase scale:

Area	Files	Approx. LOC (largest files)
Server routes	4	`chat.js` 1,844 · `dashboard.js` 1,432 · `feedback.js` 96 · `auth.js` 78
Server lib/util	9	`crawler.js` 569 · `logger.js` 434 · `rag-engine.js` 432 · `helpers.js` 283 · `message-filter-rules.js` 232
Server core	2	`index.js` 299 · `message-filter.js` 412 (+ `data/message-filter-rules.json` , 78 rules)
Server tests	16 files	51 test cases
Client components	6	`Dashboard.jsx` 1,986 · `ChatWidget.jsx` 994 · `ChatInput.jsx` , `WidgetMenu.jsx` , `App.jsx` 55
Client hooks/utils	4	`useChatSession.js` , `useChatFeedback.js` , `api.js` , `chatFormatting.js`
Documentation	8 files under `server/docs/`	Architecture, API, config, logging, message-filter, RAG, testing — genuinely thorough

2. Architecture

```
Browser (widget iframe / dashboard SPA)
        |
        v
Express server (server/index.js)  — helmet, CORS allowlist, rate limiting, ban enforcement
        |
        +-- /api/auth       -> routes/auth.js           (JWT login, PBKDF2 password check)
        +-- /api/chat       -> routes/chat.js            (1,844 lines: classification, message-filter,
        |                                                 RAG context, Gemini call, safety heuristics,
        |                                                 conversation lifecycle, logging)
        +-- /api/feedback   -> routes/feedback.js       (response/session feedback logging)
        +-- /api/*          -> routes/dashboard.js       (1,432 lines: prompt editing, rule CRUD,
                                                          rollback snapshots, blocked-response review,
                                                          feedback summaries, temp-ban management) [auth-gated]
```

Request flow for `POST /api/chat` (see `server/docs/architecture.md` for the authoritative version): sanitize input → check active ban → Gemini classifies topic/intent → deterministic safety heuristics can override the classifier for personalized legal/tax/financial questions → message-filter engine checks `block` / `respond` / `replace` rules → RAG context is fetched and appended to the system prompt → Gemini generates the reply (with one retry against `GEMINI_SECONDARY_MODEL` on a 503) → reply is post-processed (length cap, advisory-referral formatting) → the turn is logged to a daily JSONL file → response returned.

Persistence is entirely flat-file, not a database:

Store	File	Written by
Dashboard accounts	`server/data/dashboard-accounts.json`	`lib/dashboard-accounts.js` (atomic write)
Message-filter rules	`server/data/message-filter-rules.json`	`lib/message-filter-rules.js` (plain write)
Temporary bans	`server/data/ban-store.json`	`lib/ban.js` (plain write)
Vector store (RAG)	`server/data/vector-store.json`	`scripts/crawl.js`
System prompt	`server/system-prompt.txt`	`routes/dashboard.js` (plain write)
Audit/response log	`server/logs/ai-responses-MM-DD-YYYY.jsonl`	`util/logger.js` (append-only, daily rotation)
Rollback snapshots	`server/data/rollback/**`	`routes/dashboard.js`

This is a reasonable choice for a single-instance, low-write-volume internal tool, and it keeps the whole system auditable in git-adjacent flat files. The tradeoffs (no locking on most stores, no shared state across multiple server processes) are covered in §7 and §8.

3. What's Good

This codebase does a number of things noticeably better than average for its size:

1. **Correctly implemented password storage.** `lib/dashboard-accounts.js` uses PBKDF2-SHA256 at 210,000 iterations, a unique random 16-byte salt per account, and `crypto.timingSafeEqual` for comparison — modern, non-homegrown cryptography, not something thrown together. Account files are also written atomically (temp file + `fs.renameSync` + `chmod 0600`) — the one store in the codebase that gets this fully right (see §7.4 for where the rest don't).
2. **Fail-fast startup guard for `JWT_SECRET`.** `index.js` refuses to boot if the secret is missing or under 16 characters, rather than silently running insecurely. This is exactly the right instinct (undermined only by a redundant fallback string discussed in §7.2).
3. **A real rollback/audit system for a tool that edits a live chatbot's brain.** Every system-prompt or message-filter-rule change through the dashboard creates a timestamped snapshot (`routes/dashboard.js`, `createRollbackSnapshot`) and an audit log entry with actor and content hash, with a UI to restore prior state. It's rare for an internal admin tool this size to have a real undo system — this one does.
4. **Defense-in-depth LLM safety routing.** `routes/chat.js` doesn't just trust the Gemini classifier's judgment about what's safe to answer. Deterministic regex heuristics (`PERSONALIZED_HIGH_RISK_PATTERN`, `PROFESSIONAL_ADVICE_TOPIC_PATTERN`) force a safe "talk to an advisor" referral for personalized legal/tax/financial/licensing questions regardless of what the model says, and there's a full heuristic fallback classification path for when Gemini itself is unavailable. This is a genuinely thoughtful compliance-aware design for a bot giving small-business advice.
5. **Graceful degradation everywhere on the hot path.** The entire `/api/chat` handler, and every external call inside it (classification, RAG lookup, confidence scoring, the Gemini call itself with secondary-model retry), is wrapped so that upstream failures produce a safe degraded reply instead of a 500 or a crash. Logging failures are caught and never bubble into the request path either.
6. **A genuinely production-grade caching layer in the RAG engine** (`lib/rag-engine.js`) — bounded LRU-style context cache, in-flight request de-duplication so concurrent identical queries don't trigger duplicate embedding calls, and a configurable bounded-concurrency startup preload queue. This is the most mature piece of infrastructure in the codebase.
7. **Closed-loop abuse handling.** Blocked messages and rate-limit violations feed a rolling violation window that escalates to a 24-hour temporary ban, with a dashboard surface to review/clear bans — a complete detect → escalate → ban → admin-override loop, not just a bare rate limiter (its identity-key weakness is covered in §7.1).
8. **No XSS surface in chat rendering.** Assistant messages go through `react-markdown` + `remark-gfm` with no `rehype-raw` and no `dangerouslySetInnerHTML` anywhere in the client (confirmed by full-tree search). The custom link renderer adds `rel="noopener noreferrer"` correctly.

9. **Consistent client/server validation limits, enforced on both sides.** Chat message length (400 chars), feedback comment length (1000 chars), and system prompt length (8000 chars) match exactly between `client/src` and `server/routes` — genuine defense-in-depth, not just client-side theater.
10. **Bearer-token dashboard auth (not cookies)** — inherently CSRF-immune, a good architectural choice for an admin API.
11. **Thorough existing documentation.** `server/docs/` (architecture, API reference, configuration, logging schema, message-filter rules, RAG/crawling, testing/operations) is unusually complete for a project this size and was clearly kept up to date as features shipped. This report builds on top of it rather than duplicating it.
12. **Accessible custom controls where it counts.** The chat panel's resize handle is a real `<button>` with keyboard support (arrow keys, Shift for larger steps, Home to reset) and a descriptive `aria-label` — better than most mouse-only implementations. `aria-live` regions are used consistently for status/error messaging throughout the widget.

---

4. Security Findings

Findings are ranked by severity. File:line references are accurate as of this review; line numbers will drift as the code changes.

4.1 Critical

C1 — Live credential committed to git (see §0). A real dashboard account password is recoverable from git history and matches an active account. Not theoretical — cryptographically confirmed during this review.

C2 — Regex denial-of-service (ReDoS) surface on the `/api/chat` hot path. `routes/dashboard.js`'s rule-creation endpoint accepts an arbitrary admin-supplied regex pattern and only validates that it *compiles* — there is no catastrophic-backtracking check. That compiled rule is then run synchronously against every incoming chat message in `message-filter.js`'s `ruleMatches()`, on Node's single main thread. Critically, **the incoming chat `message` field has no length limit enforced before it reaches the filter** — the only cap is Express's global 1 MB JSON body limit (`index.js:165`); `MAX_MESSAGE_CHARS = 400` in `chat.js` is only applied to history truncation, not the live message. A single poorly-written rule combined with a large attacker-controlled message can block the event loop long enough to take the whole single-process server down for every concurrent user. **Fix:** enforce a message length cap (e.g. 400–2000 chars) before filtering, and validate new regex rules against a complexity/backtracking check (or a matcher with a bounded execution budget) before they're saved.

4.2 High

H1 — Ban and rate-limit identity is trivially spoofable. `index.js` sets `app.set('trust proxy', true)`, trusting the entire `X-Forwarded-For` chain unconditionally, and `lib/ban.js`'s `getRequestBanIdentity()` takes the **first** (client-supplied, least-trustworthy) entry of that header as the source IP used to key both rate limiting and the 24-hour ban store. Any client can set an arbitrary `X-Forwarded-For` value per request to get a fresh ban key every time, completely bypassing both the 10-req/min chat rate limit and the ban-on-repeated-violations system. **Fix:** set `trust proxy` to a specific hop count or a proxy IP allowlist matching your actual deployment topology, and derive the ban/rate-limit key from `req.ip` (which respects the `trust proxy` setting correctly) rather than reading the raw header directly.

H2 — `RELEASE_MODE` unset silently disables ban enforcement for message-filter violations. `routes/chat.js:1542` only calls `applyTempBan()` `if (shouldBan && env.RELEASE_MODE === 'production')`. `.env.example` documents `RELEASE_MODE` as optional ("leave unset for development mode"), and nothing in the deployment checklist in `server/docs/testing-and-operations.md` calls out that this variable is required for the ban system to function on the filter-violation path. An operator who deploys without explicitly setting `RELEASE_MODE=production` — easy to miss on many PaaS/Docker setups — gets a chatbot that logs blocks but **never actually bans repeat abusers** for content violations, even though the rate-limiter's own ban path (`index.js`) has no such gate and works regardless. This inconsistency between the two ban-triggering code paths is very easy to miss during an incident review. **Fix:** either make both ban paths respect the same gate consistently, or (better) default to enforcing bans unless explicitly disabled, and add this to the deployment checklist.

H3 — Hardcoded JWT fallback secret duplicated in two places. `routes/auth.js` signs and verifies tokens with `process.env.JWT_SECRET || 'dev_secret_change_me'` in two separate spots. Today this is unreachable because `index.js` fail-fasts at startup if `JWT_SECRET` is missing (see §3, item 2) — but it's a latent full-authentication-bypass if `auth.js` is ever imported outside that startup guard (a test harness, a serverless entry point, a future refactor). A hardcoded fallback secret checked into source control should never exist, reachable or not. **Fix:** remove the fallback; let `jwt.sign` / `jwt.verify` throw if the secret is somehow unset, rather than silently succeeding with a public string.

4.3 Medium

M1 — `server/data/dashboard-accounts.json` is tracked in git. Beyond the specific incident in §0, committing this file at all means every account's username, role, and password hash (salted PBKDF2, but still crackable offline against weak passwords) ends up in permanent git history on every account creation. **Fix:** move this file out of version control (add to `.gitignore`, point `DASHBOARD_ACCOUNTS_PATH` at a non-tracked/mounted location per the existing docs' own recommendation), and purge the currently-tracked copy from history once passwords are rotated.

M2 — CORS private-network allowlist is active unconditionally, not gated by environment. `index.js`'s `privateNetworkOriginPattern` (matches any `localhost` / `10.x` / `192.168.x` / `172.16–31.x` origin) is commented as being "for dev/test" but has no actual `RELEASE_MODE` or `NODE_ENV` check — it's

live in production too, combined with `credentials: true`. Impact is softened because dashboard auth uses a Bearer header rather than cookies, but it still needlessly widens the trusted-origin surface for the unauthenticated `/api/chat` and `/api/feedback` endpoints in production. **Fix:** gate the private-network allowlist behind a non-production check.

M3 — `/api/feedback` has no rate limiter. `index.js` applies dedicated limiters to `/api/auth`, `/api/chat`, and `/api/dashboard/*`, but `app.use('/api/feedback', feedbackRouter)` has none. This endpoint is unauthenticated and does a synchronous file append on every call — an attacker can flood it to grow the daily audit log unbounded and add I/O load with zero throttling. **Fix:** apply the same or a lighter rate limiter to `/api/feedback`.

M4 — Message-filter rules are recompiled from disk on every chat message, with no cache. `message-filter.js`'s `getRules()` re-reads `data/message-filter-rules.json` from disk and re-compiles all 78 rules' regexes on **every** incoming chat message — no in-memory cache, no mtime-based invalidation, unlike the RAG engine which caches aggressively. This adds avoidable latency and I/O to every request and amplifies the blast radius of C2. **Fix:** cache the compiled rule set in memory, invalidating on file mtime change or on dashboard rule mutation.

M5 — Read-modify-write races on most JSON "database" files. `lib/ban.js`, `lib/message-filter-rules.js`, and the system-prompt/rollback writers in `routes/dashboard.js` all write via plain `fs.writeFileSync` directly to the destination path — no temp-file-plus-rename, no locking. Two concurrent dashboard edits (or a crash mid-write) can lose an update or corrupt a store. `lib/dashboard-accounts.js` already demonstrates the correct atomic pattern elsewhere in the same codebase — it just isn't applied uniformly. **Fix:** extract the atomic-write helper from `dashboard-accounts.js` and reuse it for every JSON store.

M6 — No shared state across horizontally-scaled instances. Temporary bans (`lib/ban.js`'s in-memory `Map`, reloaded from disk only once at process start), and all of `routes/chat.js`'s conversation-lifecycle tracking (`conversationTopics`, `sessionConversationIds`, `activeConversations`, `invalidatedConversationIds`) live entirely in per-process memory. If this is ever deployed as more than one Node process/instance behind a non-sticky load balancer, bans won't be enforced consistently and conversation-expiry/active-state will behave inconsistently depending on which instance handles a given request. Not an issue for a single-instance deployment (today's likely setup), but a real blocker the moment horizontal scaling is on the table, and it isn't called out anywhere in the current ops docs.

M7 — Dashboard 401 (expired-token) handling is inconsistent across the admin UI. Roughly half of `Dashboard.jsx`'s authenticated fetches correctly detect a 401 and call `logout()`; the rest (rule add/list, rollback list/restore, rule delete/reorder, feedback summary) do not, and treat an expired session the same as any other generic failure. Once a dashboard operator's JWT expires mid-session, some panels correctly bounce them to the login screen while others just show a stuck, unhelpful error — the stale token stays in `sessionStorage` with no obvious recovery path besides finding the manual "Log Out" button. **Fix:** centralize the fetch wrapper so every authenticated call shares one 401-handling code path.

4.4 Low

- **L1 — Stale `DASHBOARD_PASSWORD=change_me` in `server/.env.example`.** This variable is documented but never read anywhere in the code; real auth is entirely account-based (`lib/dashboard-accounts.js`). An operator could reasonably set this and assume it does something. Remove it from the example file.
- **L2 — `ngrok-skip-browser-warning` header set unconditionally on every response**, and `client/vite.config.js` hardcodes a specific developer IP address and a personal ngrok tunnel hostname in `server.allowedHosts` . Both are committed dev/demo artifacts that leak internal test infrastructure details into source control and will silently stop working once that tunnel changes. Move to environment variables.
- **L3 — `GET /api/health` is unauthenticated and returns `rag.getStatus()`** , which can include `lastInitError` text (potentially a filesystem path or parse error). Minor information disclosure; common practice, low severity.
- **L4 — No max length enforced on dashboard rule fields** (`regexPattern` , `keywords` , `reply`) server-side, inconsistent with the length caps applied to the system prompt and feedback comments elsewhere in the same route file. Low severity since this endpoint is authenticated (trusted operators only).
- **L5 — Legacy `vm.runInNewContext` fallback in `lib/message-filter-rules.js`** for parsing the old `const RULES = [...]` rule-file format. `vm` is not a real sandbox in Node.js. Not reachable via any current dashboard-writable path (the dashboard always writes valid JSON), but it's a latent code-execution-shaped smell that should be deleted once the legacy format is no longer needed.
- **L6 — Font Awesome loaded from a third-party CDN in `client/index.html` with no Subresource Integrity (`integrity`) attribute.** Low risk (it's a stylesheet from a well-known CDN), but easy to harden.

---

5. Bugs and Defects

These are functional bugs, independent of security severity.

1. **Test suite does not pass cleanly in this checkout — mostly one root cause.** Running `npm test --prefix server` currently reports **42 passed / 9 failed** out of 51 tests. Eight of the nine failures (`auth-feedback-routes` , `dashboard-audit` , `dashboard-conversation-context` , `dashboard-feedback-summary` , `dashboard-rollback` , `dashboard-rule-order` , `dashboard-sandbox` , `rag-startup`) all trace back to a single corrupted dependency: `node_modules/semver` (a transitive dependency of `jsonwebtoken`) is missing `functions/truncate.js` , breaking every test file that imports anything requiring JWT verification. This looks like an incomplete/corrupted `npm install` in this specific working copy rather than a codebase defect — **run `rm -rf server/node_modules && npm install --prefix server`** to confirm it clears up, and consider committing a `package-lock.json` integrity check

(or CI, see §8) so this class of problem is caught before merge rather than discovered by whoever runs tests next.

2. **The ninth failure is a genuine, code-level Windows-portability bug.** `server/tests/evaluate-log-responses.test.js` asserts `buildDatedLogPath()` 's output against a hardcoded forward-slash regex (`/server\/logs\/response-evaluations-...\.jsonl$/`), but the implementation builds the path with `path.join()` , which produces backslash-separated paths on Windows. The test fails on any Windows dev machine even with a healthy `node_modules` . **Fix:** normalize separators before asserting, or compare path segments instead of a literal string.
3. `Dashboard.jsx` **has an orphaned "conversation context" feature that fires wasted network calls.** Clicking "Inspect" on a blocked-response card calls `loadConversationContext()` , which does a real `GET /api/dashboard/conversation-context` — but the resulting `contextTurns` / `contextSummary` / `contextNotes` / `contextStatus` / `contextError` state is never read anywhere in the render tree. Every inspection click fires and discards a network request. There's also a dead, never-called helper (`_removeContextTurnCard`) whose underscore-prefixed name suggests a half-finished removal that ESLint's unused-var pattern silently allows to persist. This looks like a feature that was partially ripped out; either finish removing it or finish wiring it up.
4. **Save button doesn't gate on the system-prompt over-limit state.** `Dashboard.jsx` shows a red "over limit" indicator once the system prompt exceeds 8,000 characters, but the Save button's `disabled` condition doesn't include that state — an operator can click Save anyway and only discover the failure after a round trip to the server.
5. **No focus trap on the two** `role="dialog" aria-modal="true"` **overlays** (the onboarding disclaimer in `ChatWidget.jsx` and the inspection modal in `Dashboard.jsx`). Neither moves focus into the dialog on open, and neither traps Tab navigation inside it — a real violation of the ARIA modal-dialog contract for keyboard/screen-reader users, not just a nice-to-have.
6. **Widget open/close doesn't manage focus.** Opening the chat panel doesn't move focus into it; closing it doesn't return focus to the launcher bubble. Screen-reader/keyboard users lose their place.
7. **Chat** `<textarea>` **has no accessible label**, relying solely on `placeholder` , which isn't a reliable name source for assistive tech.
8. **Stale/misplaced file-header doc comment.** `client/src/ChatWidget.jsx` still carries a large header comment describing `App.jsx` 's routing responsibilities (`/` vs `/dashboard`) — leftover from before that logic was extracted into the actual, much smaller `App.jsx` . Confusing for anyone onboarding onto this file.
9. **Several** `Dashboard.jsx` **error handlers discard the actual error message** (`loadPageUsage` , `loadFeedbackSummary` just set a generic "Unable to load..." with no detail), inconsistent with other handlers in the same file that do surface `error.message` . Makes production debugging harder than it needs to be.
10. **No client-side length caps on the dashboard rule-editor fields** (rule ID, regex pattern, reply text), unlike the system-prompt and feedback forms which do enforce and display limits — an inconsistency within the same UI.

6. Test Suite & Dependency Audit

6.1 Server tests

```
npm test --prefix server
ℹ tests 51
ℹ pass 42
ℹ fail 9
```

Root causes: 8 failures from one corrupted `node_modules/semver` install in this checkout (§5.1); 1 genuine Windows path-separator bug in a test assertion (§5.2). Test **coverage itself** (per `server/docs/testing-and-operations.md`) is solid on paper — chat routing, dashboard rule ordering, feedback summaries, auth, bans, sanitization helpers, and message-filter behavior all have dedicated suites — but there is **no CI workflow** in the repository (`.github/workflows` does not exist), so nothing currently enforces that these tests pass before merge. That's the single biggest process gap found in this review: none of the issues in this report would have been *prevented* by CI, but a CI gate would have caught the current 9 failing tests before they became "future someone's problem."

6.2 Dependency audit

Server (`npm audit --prefix server`): 3 vulnerabilities (1 low, 2 high) — `brace-expansion` (ReDoS/DoS), `undici` (multiple advisories: TLS bypass, header injection, cache poisoning), `body-parser` (DoS via invalid limit value). All have fixes available via `npm audit fix`.

Client (`npm audit --prefix client`): 5 high-severity vulnerabilities — `brace-expansion`, `js-yaml` (quadratic-complexity DoS), `nanoid`, `postcss` (path traversal in sourcemap loading), `vite` / `launch-editor` (NTLMv2 hash disclosure on Windows, `server.fs.deny` bypass). These are all **build-tooling/dev-dependency chain** issues (Vite, PostCSS, etc.) rather than runtime dependencies shipped to the browser, so production exposure is lower than the "5 high" count suggests at a glance — but they should still be patched, and `npm audit fix` resolves most of them without a major version bump.

Note: `client/node_modules` did not exist in this checkout before this review (dependencies had never been installed here) — worth confirming this isn't the norm for how contributors set up the project, since it means client lint/build has likely never been run in this particular workspace/worktree until now.

7. Potential Blockers for Production / Scale

Ranked roughly by how soon each would bite:

1. **The credential exposure in §0** — blocks nothing technically, but is an active risk right now, today, independent of any deployment plans.
2. **Ban/rate-limit bypass via spoofed `X-Forwarded-For` (§4.2, H1)** — if this server is ever reachable directly (not strictly behind a trusted reverse proxy that overwrites the header), the entire abuse-prevention system is a no-op against a motivated attacker.
3. `RELEASE_MODE` **footgun (§4.2, H2)** — silently disables ban enforcement for content violations unless an operator remembers a specific, easy-to-miss environment variable.
4. **No horizontal scaling support (§4.3, M6)** — bans, rate-limit violation windows, and conversation lifecycle state are all per-process. Scaling this beyond one instance (a very plausible next step if traffic grows) requires moving that state into a shared store (Redis, a database, or similar) first.
5. **No CI pipeline** — nothing blocks a broken build or a failing test suite from being merged. Given the size of `chat.js` and `dashboard.js`, regressions are easy to introduce silently.
6. **Regex DoS surface (§4.1, C2)** — a single bad dashboard-authored rule (even created with good intentions by a non-engineer admin) can degrade or take down the whole server for every user, since it's a single Node process with no request-level execution timeout.
7. **Dependency vulnerabilities (§6.2)** — none are currently known to be exploited in this app's specific usage pattern, but they're free to fix (`npm audit fix`) and shouldn't linger.
8. **No self-service password rotation** — dashboard accounts can only be created via a CLI script (`scripts/create-dashboard-account.js`); there's no supported way to change a password without recreating the account. This makes the remediation in §0 more awkward than it should be, and will keep being awkward for the next rotation too.

None of these are architecture-breaking — they're the normal list of things that separate "a working internal tool" from "a tool that's been through a hardening pass."

---

8. Recommended Next Steps

Do now (this week): - Rotate `mark.barnett`'s dashboard password and `JWT_SECRET` (§0). - Stop tracking `server/data/dashboard-accounts.json` in git; point it at a non-tracked path. - Add a message length cap before message-filter evaluation in `routes/chat.js` (closes most of C2's blast radius on its own). - Fix the `trust proxy` / `X-Forwarded-For` handling in `index.js` + `lib/ban.js` (H1). - Decide the intended behavior for `RELEASE_MODE` and either fix the gate or fix the docs so they agree (H2). - Remove the hardcoded `dev_secret_change_me` fallback in `routes/auth.js` (H3).

Do soon (this month): - Stand up a minimal CI workflow: `npm test --prefix server`, `npm run lint --prefix client`, `npm run build --prefix client` on every PR. This alone would have caught 9 of this review's findings automatically going forward. - Extract the atomic-write pattern from `lib/dashboard-accounts.js` into a shared helper and apply it to `lib/ban.js`, `lib/message-filter-rules.js`, and the system-prompt writer (M5). - Add caching (with invalidation) to `message-filter.js`'s rule loading (M4). -

Add a rate limiter to `/api/feedback` (M3). - Centralize the dashboard's fetch/401-handling logic so session expiry behaves consistently everywhere (M7). - Fix the Windows path-separator test and reinstall a clean `server/node_modules` to get the suite fully green (§5.1–5.2). - Run `npm audit fix` in both `server/` and `client/`.

Plan for (this quarter): - If traffic/scale plans call for more than one server instance, design a shared store (Redis or similar) for bans, rate-limit windows, and conversation lifecycle state before scaling out (M6). - Add a supported password-rotation path for dashboard accounts, not just creation. - Accessibility pass on the two modal dialogs (focus trap + initial focus) and widget open/close focus management (§5, items 5–7). - Consider splitting `routes/chat.js` (1,844 lines) and `routes/dashboard.js` (1,432 lines) into smaller modules (classification, safety heuristics, conversation lifecycle, and route handlers as separate files) — not urgent, but both are past the point where a newcomer can hold the whole file in their head, and it's only going to grow.

---

9. Appendix

9.1 Existing documentation map

This report is a point-in-time cross-cutting review; for day-to-day reference, `server/docs/` remains the source of truth and is genuinely well maintained:

- `server/docs/architecture.md` — request flow, shared state, error-handling philosophy
- `server/docs/configuration.md` — full environment variable reference
- `server/docs/api.md` — endpoint-by-endpoint API reference
- `server/docs/logging-and-dashboard-data.md` — JSONL log schema and redaction rules
- `server/docs/message-filter-and-safety.md` — rule fields, evaluation order, ban mechanics
- `server/docs/rag-and-crawling.md` — vector store, crawling, RAG tuning
- `server/docs/testing-and-operations.md` — test commands, deployment checklist, troubleshooting

9.2 Review methodology

This report was produced by reading the full source of every server route/lib/util module and every client component/hook/util, running the server test suite and client lint, running `npm audit` on both workspaces, inspecting git history for the configuration-doc diff, and cryptographically verifying the leaked-credential finding against the committed account store. It reflects the codebase as of commit `732500b` on this branch.